DS Docker Security
Best Practices 2026

개요
1. 개요 및 위협 환경
2. 우선순위 요약

보안 상세 (OWASP)
컨테이너 이미지 보안 Critical
최소 권한 실행 원칙 Critical
시크릿 관리 High
네트워크 격리 High
런타임 모니터링 Medium
리소스 제한 Medium
불변 컨테이너 Medium
보안 기본값 High
호스트 OS 보안 High
공급망 보안 Medium

부록
4. 점검 체크리스트
5. 참고 문헌

— Security Reference for Engineers & Developers

Docker Security Best Practices

컨테이너 환경 보안 모범 사례 가이드 · 2026

`OWASP Docker Top 10` `CIS Docker Benchmark v1.8.0` `NIST SP 800-190`

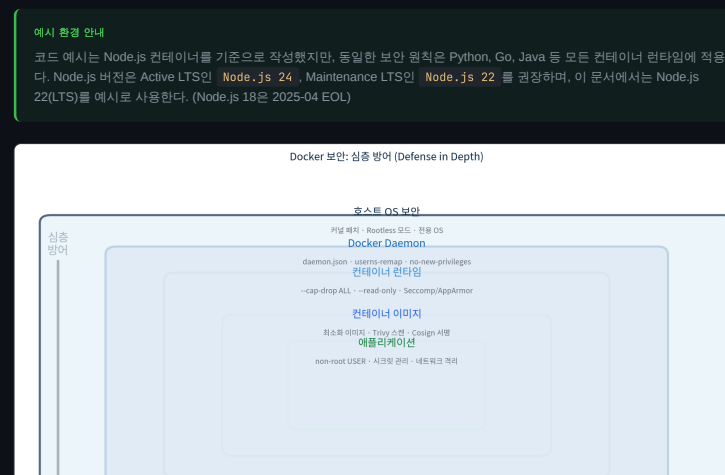
---

01 개요 및 위협 환경

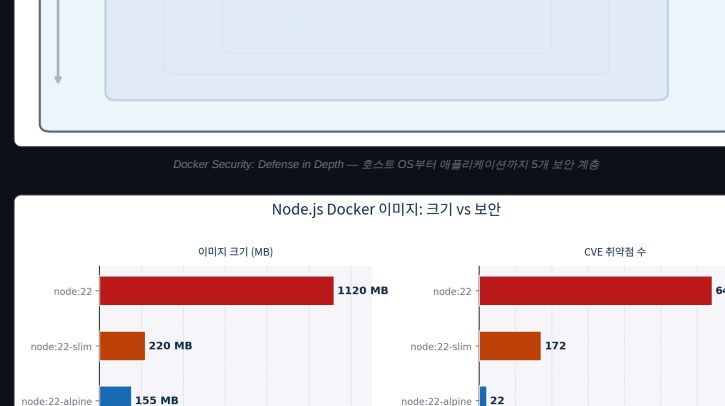
Docker 컨테이너는 호스트 OS 커널을 공유한다. VM과 달리 컨테이너 내 설정 하나가 호스트 전체 또는 동일 호스트의 다른 컨테이너에 영향을 미칠 수 있다.

이 문서는 OWASP Docker Top 10 D01~D10 전 항목을 위험도와 구현 비용 기준으로 재구성하여, 즉시 적용 가능한 대응 방안을 단계별로 제시한다. Sysdig 2024 Cloud-Native Security Report[7]에 따르면, 컨테이너 환경 보안 사고의 주요 원인은 잘못된 설정(misconfiguration)과 과도한 권한 부여이며, 이는 이 문서에서 다루는 D01~D08 항목과 직접 연결된다.

> **예시 환경 안내**
> 코드 예시는 Node.js 컨테이너를 기준으로 작성했지만, 동일한 보안 원칙은 Python, Go, Java 등 모든 컨테이너 런타임에 적용된다. Node.js 버전은 Active LTS인 `Node.js 24`, Maintenance LTS인 `Node.js 22`를 권장하며, 이 문서에서는 Node.js 22(LTS)를 예시로 사용한다. (Node.js 18은 2025-04 EOL)


Docker 보안: 심층 방어 (Defense in Depth)
심층 방어
호스트 OS 보안
커널 패치 · Rootless 모드 · 전용 OS
Docker Daemon
daemon.json · userns-remap · no-new-privileges
컨테이너 런타임
--cap-drop ALL · --read-only · Seccomp/AppArmor
컨테이너 이미지
최소화 이미지 · Trivy 스캔 · Cosign 서명
애플리케이션
non-root USER · 시크릿 관리 · 네트워크 격리

Docker Security: Defense in Depth — 호스트 OS부터 애플리케이션까지 5개 보안 계층



Node.js Docker 이미지 유형별 크기 및 CVE 수 비교 (Snyk & Docker Scout, 2024)

터미널 — docker images + trivy 비교 CVE 97% 감소

```
$ docker images --format "table {{.Repository}}:{{.Tag}}        {{.Size}}"
REPOSITORY:TAG            SIZE
node:latest               1.09GB    ← 권장하지 않음
node:22-alpine            135MB     ← 권장 (87% 감소)
node:22-slim              218MB     ← 네이티브 모듈 필요 시

$ trivy image --severity HIGH,CRITICAL node:latest 2>/dev/null | tail -3
Total: 642 (HIGH: 378, CRITICAL: 264)

$ trivy image --severity HIGH,CRITICAL node:22-alpine 2>/dev/null | tail -3
Total: 22 (HIGH: 18, CRITICAL: 4)

$ trivy image --severity HIGH,CRITICAL gcr.io/distroless/nodejs22-debian12 2>/dev/null | tail -3
Total: 3 (HIGH: 3, CRITICAL: 0)
```

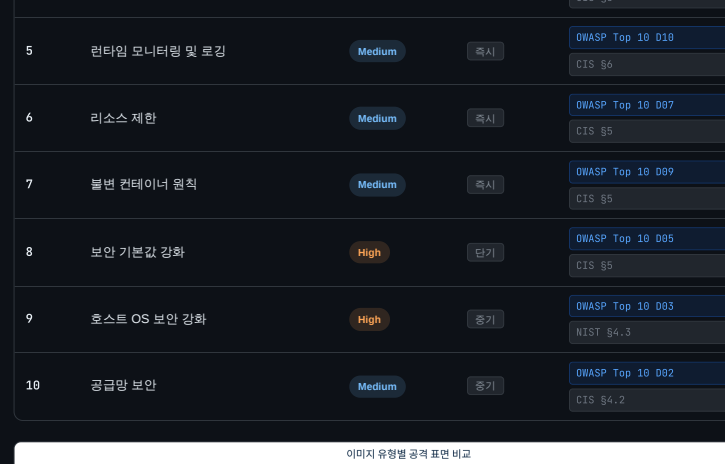
> **핵심 포인트**
> 이미지 선택 하나만으로 CVE를 **642개에서 22개로** 줄일 수 있다 (97% 감소). 보안 투자 대비 효과가 가장 높은 첫 번째 조치다.

02 보안 고려사항 우선순위 요약

위협도와 구현 비용을 기준으로 OWASP Docker Top 10 전 항목을 정렬했다. 구현 시점 기준 — 즉시: Dockerfile/compose 수정, 단기: 파이프라인·정책 변경, 중기: 인프라 수준 변경

> **우선순위 선정 기준**
> **Critical** = 호스트 root 탈취 또는 전체 컨테이너 침해 가능 (CVSS 9.0+급 취약점 직접 연결) | **High** = 기밀정보 유출 또는 격리 우회 경로 존재 | **Medium** = 가용성 저하 또는 탐지 지연. 동일 위험도 내에서는 구현 비용이 낮은 항목을 우선 배치했다.

순위	보안 고려사항	위험도	구현 시점	표준 참조
1	컨테이너 이미지 보안	Critical	즉시	OWASP Top 10 D08 CIS §4
2	최소 권한 실행 원칙	Critical	즉시	OWASP Top 10 D01 CIS §4.1
3	시크릿 및 민감정보 관리	High	즉시	OWASP Top 10 D06 CIS §4
4	네트워크 격리 및 접근 제어	High	즉시	OWASP Top 10 D04 CIS §5
5	런타임 모니터링 및 로깅	Medium	즉시	OWASP Top 10 D10 CIS §6
6	리소스 제한	Medium	즉시	OWASP Top 10 D07 CIS §5
7	불변 컨테이너 원칙	Medium	즉시	OWASP Top 10 D09 CIS §5
8	보안 기본값 강화	High	단기	OWASP Top 10 D05 CIS §5
9	호스트 OS 보안 강화	High	중기	OWASP Top 10 D03 NIST §4.3
10	공급망 보안	Medium	중기	OWASP Top 10 D02 CIS §4.2



이미지 유형별 공격 표면(Attack Surface) 비교 — node:latest = 100 기준 (Snyk, 2024)

> **핵심 포인트**
> 이미지 유형 변경만으로 공격 표면을 92% 감소시킬 수 있다. 패키지 수 84%, 네트워크 노출 벡터 55%, 공격 가능 벡터 92% 감소.

03 보안 고려사항 상세

#1 순위 `OWASP Top 10 D08` 컨테이너 이미지 보안 — Critical CIS §4

공격자는 대부분 패치되지 않은 CVE(Common Vulnerabilities and Exposures)를 통해 초기 침투 경로를 확보한다. 이미지 선택과 스캔 자동화가 컨테이너 보안의 첫 번째 방어선이 된다.

> **실제 사례**
> 2021년, Docker Hub에서 azurecr를 사칭한 이미지 수십 개가 수천 회 다운로드됐다. 이미지 내부에 XMRig 크립토마이너가 포함되어 있었으나, 이미지 서명 검증만 적용되어 있었어도 배포 단계에서 차단할 수 있었다.

1. 최소화 이미지 + 버전 고정 — 즉시
2. Trivy CI/CD 통합 (빌드 차단) — 즉시
3. 이미지 서명 검증 (DCT / Cosign) — 즉시
4. Distroless 이미지 전환 — 단기

1순위 최소화 이미지 + 버전 고정 [즉시]

`node:latest` 처럼 태그를 고정하지 않으면 빌드 시마다 다른 이미지가 사용되고, Debian 기반 전체 패키지가 포함되어 CVE 642개를 탑재한 채로 운영에 들어가게 된다.[6] `node:22-alpine` 으로 변경하면 CVE가 22개 수준으로 감소하고 이미지 크기도 70% 줄어든다.[6]

DOCKERFILE ❌ Bad Practice

```dockerfile
FROM node:latest          # 642개 CVE, 매 빌드마다 이미지 변경됨
```

DOCKERFILE ✅ Best Practice

```dockerfile
FROM node:22-alpine          # Node LTS + Alpine 경량 이미지
FROM node:22-alpine@sha256:a1b2c3d4...          # digest 고정으로 태그 변조까지 방어
```

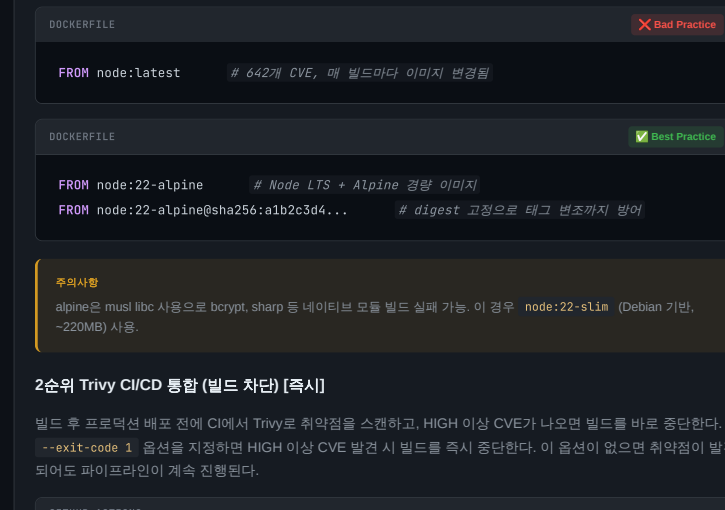
> **주의사항**
> alpine은 musl libc 사용으로 bcrypt, sharp 등 네이티브 모듈 빌드 실패 가능. 이 경우 `node:22-slim` (Debian 기반, ~220MB) 사용.

2순위 Trivy CI/CD 통합 (빌드 차단) [즉시]

빌드 후 프로덕션 배포 전에 CI에서 Trivy로 취약점을 스캔한다. HIGH 이상 CVE가 나오면 빌드를 바로 중단한다. `--exit-code 1` 옵션을 지정하면 HIGH 이상 CVE 발견 시 빌드를 즉시 중단한다. 이 옵션이 없으면 취약점이 발견되어도 파이프라인이 계속 진행된다.

GITHUB ACTIONS

```yaml
- run: |
    trivy image \
      --exit-code 1 \          # 취약점 발견 시 파이프라인 중단
      --severity HIGH,CRITICAL \          # 이 심각도 이상만 검사
      --ignore-unfixed \          # 패치 미존재 CVE 제외
      myapp:${{ github.sha }}
```



CI/CD 파이프라인 보안 통합 — Trivy 스캔에서 HIGH/CRITICAL CVE 발견 시 자동 차단

> **참고 — 스캐너 선택**
> Trivy 외에도 Grype(Anchore), Snyk Container, Docker Scout 등 대안이 있다. 이 문서에서는 오픈소스이면서 CI 통합이 쉽고 SBOM(Software Bill of Materials, 소프트웨어 구성 명세) 생성까지 지원하는 Trivy를 예시로 사용한다.

Github Actions — trivy-scan BUILD FAILED — CVE 발견

```
2024-11-01T09:12:33.401Z INFO  Vulnerability scanning is enabled
2024-11-01T09:12:33.402Z INFO  Secret scanning is enabled
2024-11-01T09:12:35.112Z INFO  Detected OS: alpine 3.19.1

myapp:a3f91b2 (alpine 3.19.1)

Total: 3 (HIGH: 2, CRITICAL: 1)

┌──────────┬────────────────┬──────────┬───────────────────────────┐
│ Library  │ Vulnerability  │ Severity │ Installed / Fixed Version │
├──────────┼────────────────┼──────────┼───────────────────────────┤
│ openssl  │ CVE-2024-5535  │ CRITICAL │ 3.1.4-r5 → 3.1.6-r0       │
│ libexpat │ CVE-2024-45492 │ HIGH     │ 2.6.2-r0 → 2.6.3-r0       │
│ busybox  │ CVE-2023-42364 │ HIGH     │ 1.36.1-r15 → 1.36.1-r19   │

FAIL  exit code 1 — HIGH/CRITICAL CVE 발견으로 배포 차단됨
##[error]Process completed with exit code 1.
```

3순위 이미지 서명 검증 (DCT / Cosign) [즉시]

`DOCKER_CONTENT_TRUST=1` (DCT, Docker Content Trust)을 설정하면 이미지 pull 시 서명을 자동으로 검증하며, 서명되지 않은 이미지는 pull 단계에서 차단된다.

더 최신 방식인 Cosign은 OCI(Open Container Initiative) 레지스트리 표준을 따른다. Kubernetes Admission Webhook과 연동하면 미서명 이미지의 클러스터 배포를 정책 수준에서 차단할 수 있다.

SHELL

```
$ export DOCKER_CONTENT_TRUST=1          # 미서명 이미지 pull 거부
```

Cosign 키 생성·서명·검증 명령어는 **3.10 공급망 보안 > 3순위**에서 다룬다.

4순위 Distroless 이미지 전환 [단기]

Distroless 이미지는 앱 런타임과 의존성만 포함한다. shell·패키지 매니저·일반 리눅스 유틸리티는 제외된다. RCE(Remote Code Execution, 원격 코드 실행)가 발생하더라도 bash, curl, wget이 없어 공격자가 추가 공격 행위를 수행하기 어렵다. CVE 수도 5개 이하로 감소한다.[6]

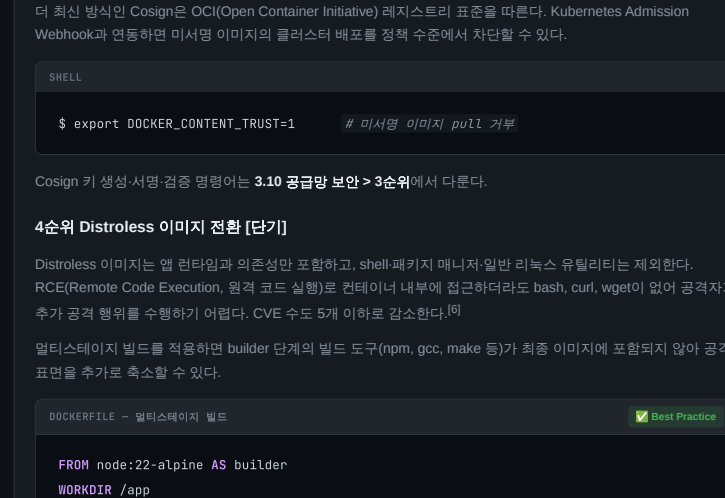
멀티스테이지 빌드를 적용하면 builder 단계에서 빌드 도구(npm, gcc, make 등)가 최종 이미지에 포함되지 않아 공격 표면을 추가로 축소할 수 있다.

DOCKERFILE — 멀티스테이지 빌드 ✅ Best Practice

```dockerfile
FROM node:22-alpine AS builder
WORKDIR /app
COPY package*.json ./
RUN npm ci --only=production
COPY . .

FROM gcr.io/distroless/nodejs22-debian12
COPY --from=builder /app /app
CMD ["app.js"]          # shell 없음, 패키지 관리자 없음, CVE 5개 이하
```

#2 순위 `OWASP Top 10 D01` 최소 권한 실행 원칙 — Critical CIS §4.1

컨테이너가 root(UID 0)로 실행되는 상태에서 탈출 취약점이 발생하면, 공격자는 즉시 호스트 root 권한을 획득한다. non-root 사용자로 실행하면 피해 범위가 컨테이너 내부로 제한된다.



Root vs Non-root 컨테이너 — CVE-2019-5736 탈출 시나리오 비교

> **실제 사례 — CVE-2019-5736 (RUNC, 2019)**
> root로 실행 중인 컨테이너 내부에서 호스트의 runc 바이너리를 덮어쓰는 방식으로 호스트 root 권한 탈취가 가능하다. Docker 18.09.2 이전 버전 전체에 영향을 미쳤다. non-root 컨테이너에서는 write 권한이 제한되어 동일한 공격 경로 적용이 어렵다.

1. Dockerfile non-root 유저 지정 — 즉시
2. 런타임 Linux capability 최소화 — 즉시
3. 권한 상승 차단 (no-new-privileges) — 즉시
4. Docker Rootless 모드 전환 — 단기

1순위 Dockerfile non-root 유저 지정 [즉시]

Docker 컨테이너는 기본적으로 root(UID 0)로 시작한다. Dockerfile에서 `addgroup` / `adduser` 로 전용 계정을 생성하고 `USER` 로 전환하면, 이후 `CMD` · `ENTRYPOINT` 가 모두 해당 계정 권한으로 실행된다.

DOCKERFILE ❌ Bad Practice

```dockerfile
FROM node:22-alpine
COPY . /app
CMD ["node", "app.js"]          # root(UID 0)로 실행
```

DOCKERFILE ✅ Best Practice

```dockerfile
FROM node:22-alpine
RUN addgroup -S appgroup && adduser -S appuser -G appgroup
COPY --chown=appuser:appgroup . /app
USER appuser          # 이후 모든 명령 appuser 권한
CMD ["node", "app.js"]
```

터미널 — 컨테이너 내 실행 유저 확인 non-root 확인

```
# ❌ USER 미지정 컨테이너
$ docker run --rm node:22-alpine whoami
root
$ docker run --rm node:22-alpine id
uid=0(root) gid=0(root) groups=0(root),1(bin),2(daemon)...

# ✅ non-root USER 지정 후
$ docker run --rm myapp whoami
appuser
$ docker run --rm myapp id
uid=100(appuser) gid=101(appgroup) groups=101(appgroup)
$ docker inspect myapp --format='{{.Config.User}}'
appuser
```

2순위 런타임 Linux capability 최소화 [즉시]

Docker 컨테이너는 기본으로 14개의 Linux capability(CAP_NET_RAW, CAP_CHOWN 등)를 보유한다.[5] `--cap-drop ALL` 로 전부 제거한 뒤, 실제로 필요한 capability만 `--cap-add` 로 명시적으로 추가하는 것을 권장한다.

SHELL

```
$ docker run \
  --cap-drop ALL \          # 14개 기본 capability 전부 제거
  --cap-add NET_BIND_SERVICE \          # 80/443 바인딩 필요 시만 추가
  --user 1001:1001 \
  myapp
```

터미널 — Linux Capability 비교 Capability 최소화 확인

```
# ❌ 기본 컨테이너 capability (14개)
$ docker run --rm node:22-alpine sh -c "cat /proc/1/status | grep CapEff"
CapEff: 00000000a80425fb
# CAP_CHOWN, CAP_DAC_OVERRIDE, CAP_FOWNER, CAP_SETGID, CAP_NET_RAW ...

# ✅ --cap-drop ALL 적용 후
$ docker run --rm --cap-drop ALL myapp sh -c "cat /proc/1/status | grep CapEff"
CapEff: 0000000000000000   ← 모든 capability 제거됨

# ✅ NET_BIND_SERVICE 하나만 추가
$ docker run --rm --cap-drop ALL --cap-add NET_BIND_SERVICE myapp sh -c "cat /proc/1/statu
s | grep CapEff"
CapEff: 0000000000000400   ← CAP_NET_BIND_SERVICE(10번)만 활성화
```

3순위 권한 상승 차단 (no-new-privileges) [즉시]

`--no-new-privileges` 는 setuid/setgid 바이너리를 통한 권한 상승을 커널 수준에서 막는다. /etc/docker/daemon.json에 전역으로 설정해 두면 모든 컨테이너에 자동 적용된다.

SHELL / DAEMON.JSON

```
$ docker run --no-new-privileges myapp

# /etc/docker/daemon.json
{ "no-new-privileges": true }
```

4순위 Docker Rootless 모드 전환 [단기]

일반 Docker 설치에서는 daemon 자체가 root로 실행된다. Rootless 모드는 daemon을 일반 유저(UID 1000) 권한으로 실행하여, daemon 취약점이 발생해도 호스트 root 권한으로 이어지지 않도록 격리한다.

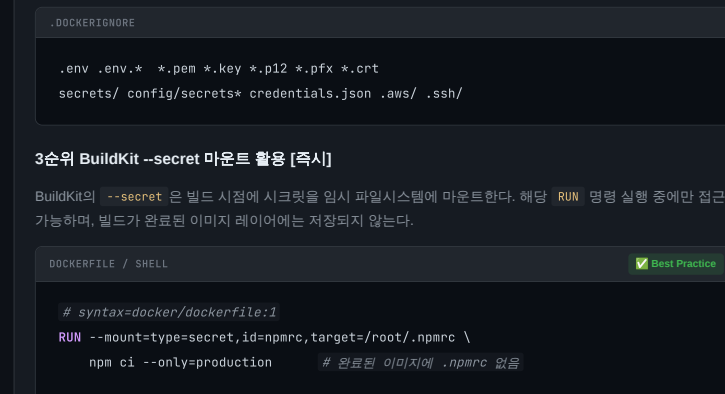
SHELL

```
$ dockerd-rootless-setuptool.sh install
$ export DOCKER_HOST=unix:///run/user/$(id -u)/docker.sock
```

> **주의사항**
> 80/443 포트 직접 바인딩 시 NET_BIND_SERVICE 필요. 권장: 앞단 Nginx + 앱 8080 포트 → `--cap-add` 없이 `--cap-drop ALL` 만으로 운영 가능.

#3 순위 `OWASP Top 10 D06` 시크릿 및 민감정보 관리 — High CIS §4

이미지 레이어는 한 번 쌓이면 변경되지 않는다. `ENV` / `ARG` 에 삽입된 시크릿은 이미지가 존재하는 한 `docker history` 로 조회할 수 있다. 상위 레이어에서 `RUN rm` 을 실행해도 이전 레이어에 기록된 데이터는 삭제되지 않는다.


이미지 레이어에 남는 시크릿 노출 경로
RUN rm으로 삭제해도 이전 레이어에 남아있음
Layer 3: RUN rm .env
Layer 2: ENV DB_PASSWORD=secret123
Layer 1: FROM node:22-alpine
docker history --no-trunc
DB_PASSWORD=secret123
⚠ 영구 노출!

이미지 레이어에 남는 시크릿 노출 경로 — RUN rm으로 삭제해도 이전 레이어에 잔존

> **실제 사례**
> 2022년 연구(Sysdig Threat Research Team): Docker Hub 공개 이미지 분석 결과 상당수에서 AWS 키, DB 패스워드 등 시크릿 발견. 대부분 ENV 또는 빌드 ARG로 주입된 것이었다.

1. ENV/ARG 시크릿 하드코딩 제거 — 즉시
2. .dockerignore 필수 설정 — 즉시
3. BuildKit --secret 마운트 활용 — 즉시
4. Docker Secrets / Vault 런타임 주입 — 단기

1순위 ENV/ARG 시크릿 하드코딩 제거 [즉시]

Dockerfile에 `ENV DB_PASSWORD=secret123` 한 줄만 있어도 그 값은 이미지 레이어에 영구 기록된다. `docker history --no-trunc` 를 실행하면 해당 값이 그대로 출력된다.

DOCKERFILE / SHELL ❌ Bad Practice

```
ENV DB_PASSWORD=secret123          # 레이어에 영구 기록
ARG NPM_TOKEN          # docker history --no-trunc 으로 노출

# 시크릿 노출 여부 확인
$ docker history --no-trunc myapp | grep -iE 'password|secret|token|key'
```

터미널 — docker history 시크릿 노출 확인 시크릿 노출 위험

```
# ❌ ENV로 시크릿을 넣은 이미지의 history 확인
$ docker history --no-trunc myapp | grep -iE 'password|secret|token|key'
sha256:a3f... /bin/sh -c #(nop) ENV DB_PASSWORD=P@ssw0rd!secret123 <missing>
sha256:b2c... /bin/sh -c #(nop) ARG NPM_TOKEN=npm_Kx8mP3qR...REDACTED      <missing>

# ✅ BuildKit --secret 사용 후 — history에 시크릿 없음
$ docker history --no-trunc myapp-secure | grep -iE 'password|secret|token|key'
(no output)
$ echo $?
1  ← grep 결과 없음 (정상)
```

2순위 .dockerignore 필수 설정 [즉시]

.dockerignore가 없으면 docker build 시 .env, *.pem, credentials.json 같은 민감 파일이 빌드 컨텍스트에 포함되어 이미지 레이어에 그대로 번들링된다.

.DOCKERIGNORE

```
.env .env.*  *.pem *.key *.p12 *.pfx *.crt
secrets/ config/secrets* credentials.json .aws/ .ssh/
```

3순위 BuildKit --secret 마운트 활용 [즉시]

BuildKit의 `--secret` 은 빌드 시점에 시크릿을 임시 파일시스템에 마운트한다. 해당 `RUN` 명령 실행 중에만 접근 가능하며, 빌드가 완료된 이미지 레이어에는 저장되지 않는다.

DOCKERFILE / SHELL ✅ Best Practice

```
# syntax=docker/dockerfile:1
RUN --mount=type=secret,id=npmrc,target=/root/.npmrc \
    npm ci --only=production          # 완료된 이미지에 .npmrc 없음

$ DOCKER_BUILDKIT=1 docker build --secret id=npmrc,src=$HOME/.npmrc .
```

4순위 Docker Secrets / Vault 런타임 주입 [단기]

Docker Swarm Secrets는 시크릿을 tmpfs(메모리 파일시스템)에 마운트해 `/run/secrets/<name>` 으로 읽게 한다. 더 정교한 환경에서는 HashiCorp Vault Agent Sidecar나 AWS Secrets Manager SDK로 런타임에 동적으로 가져오는 방식을 쓴다.

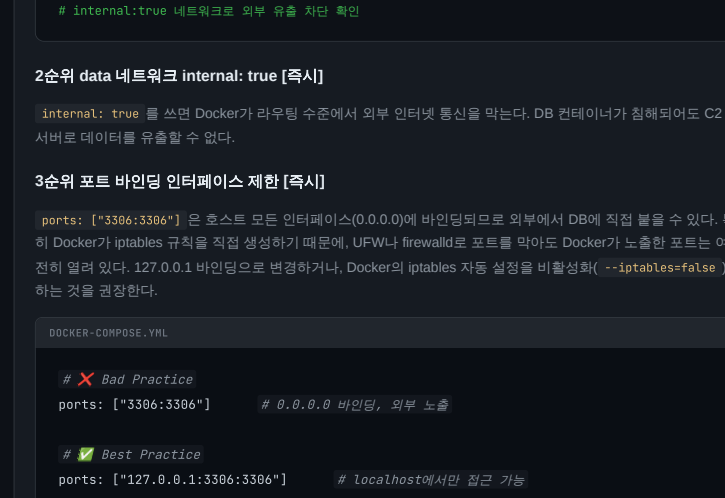
SHELL

```
$ printf 'mypassword' | docker secret create db_password -
$ docker service create --secret db_password myapp

# 앱: fs.readFileSync('/run/secrets/db_password', 'utf8').trim()
```

> **주의사항**
> Compose의 `environment:` 섹션 직접 값 작성 시 docker inspect로 조회 가능. `env_file:` 사용 + .gitignore 및 .dockerignore 양쪽에 추가 필수.

#4 순위 `OWASP Top 10 D04` 네트워크 격리 및 접근 제어 — High CIS §5

기본 bridge 네트워크는 모든 컨테이너가 동일 세그먼트에 위치한다. 웹 서버 컨테이너 하나가 침해되면 공격자가 DB·캐시·내부 API로 자유롭게 횡이동(Lateral Movement)할 수 있다.



네트워크 격리 전/후: internal 네트워크 적용 시 Lateral Movement 차단

1. 서비스 계층별 네트워크 분리 — 즉시
2. data 네트워크 internal: true — 즉시
3. 포트 바인딩 인터페이스 제한 — 즉시
4. docker.sock 마운트 금지 — 즉시

1순위 서비스 계층별 네트워크 분리 [즉시]

frontend(nginx)·app(api)·data(DB, Redis)를 3계층으로 나누면 nginx는 api와만, api는 DB/Redis와만 통신하도록 경로가 고정된다.

DOCKER-COMPOSE.YML ✅ Best Practice

```yaml
services:
  nginx: { networks: [frontend] }
  api:   { networks: [frontend, app] }          # 중간 계층
  db:    { networks: [app] }          # 외부 포트 노출 없음
  redis: { networks: [app] }

networks:
  frontend: {}
  app: { internal: true }          # 외부 인터넷 접근 완전 차단
```

터미널 — docker network 격리 확인 네트워크 격리 적용

```
$ docker network ls
NETWORK ID     NAME             DRIVER    SCOPE
a1b2c3d4e5f6   myapp_frontend   bridge    local
b2c3d4e5f6a1   myapp_app        bridge    local   ← internal:true
c3d4e5f6a1b2   bridge           bridge    local

$ docker network inspect myapp_app --format='{{.Internal}}'
true  ← 외부 인터넷 완전 차단 확인

# DB 컨테이너에서 외부 접근 시도 → 차단됨
$ docker exec myapp_db_1 curl -s --max-time 3 https://example.com
curl: (28) Connection timed out after 3001 milliseconds
# internal:true 네트워크로 외부 유출 차단 확인
```

2순위 data 네트워크 internal: true [즉시]

`internal: true` 를 쓰면 Docker가 라우팅 수준에서 외부 인터넷 통신을 막는다. DB 컨테이너가 침해되어도 C2 서버로 데이터를 유출할 수 없다.

3순위 포트 바인딩 인터페이스 제한 [즉시]

`ports: ["3306:3306"]` 은 호스트 모든 인터페이스(0.0.0.0)에 바인딩되므로 외부에서 DB에 직접 붙을 수 있다. 특히 Docker가 iptables 규칙을 직접 생성하기 때문에, UFW나 firewalld로 포트를 막아도 Docker가 노출한 포트는 여전히 열려 있다. 127.0.0.1 바인딩으로 변경하거나, Docker의 iptables 자동 설정을 비활성화(`--iptables=false`)하는 것을 권장한다.

DOCKER-COMPOSE.YML

```yaml
# ❌ Bad Practice
ports: ["3306:3306"]          # 0.0.0.0 바인딩, 외부 노출

# ✅ Best Practice
ports: ["127.0.0.1:3306:3306"]          # localhost에서만 접근 가능
```

4순위 docker.sock 마운트 금지 [즉시]

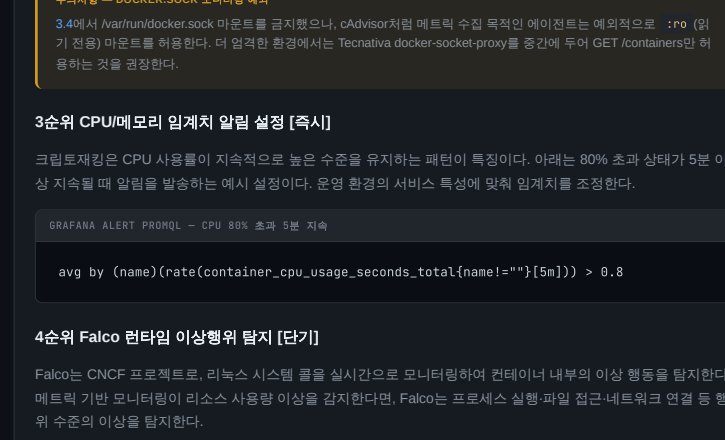
`/var/run/docker.sock` 을 마운트하면, 해당 소켓을 마운트한 컨테이너에서 호스트의 모든 컨테이너를 제어하고 root 권한으로 새 컨테이너를 실행하는 것이 가능하다. Portainer 등 관리 도구에서 반드시 필요한 경우, Tecnativa Docker Socket Proxy를 중간에 배치하여 필요한 API만 노출하는 것을 권장한다.

DOCKER-COMPOSE.YML

```yaml
# ❌ Bad Practice — 절대 금지
volumes: [/var/run/docker.sock:/var/run/docker.sock]

# ✅ Best Practice — Socket Proxy 사용
socket-proxy:
  image: tecnativa/docker-socket-proxy
  environment: { CONTAINERS: 1, POST: 0 }          # 쓰기 API 전체 차단
  volumes: [/var/run/docker.sock:/var/run/docker.sock:ro]
```

#5 순위 `OWASP Top 10 D10` 런타임 모니터링 및 로깅 — Medium CIS §6

침해는 예방만큼 신속한 탐지도 중요하다. 크립토재킹은 서비스가 정상 동작하면서 CPU만 과점유하는 형태로 진행되기 때문에, 모니터링 체계가 없으면 수 주간 감지되지 않을 수 있다.



컨테이너 모니터링 스택 — cAdvisor → Prometheus → Grafana + Falco 이상행위 탐지

> **실제 사례**
> Kubernetes 대시보드 미인증 접근 → 크립토마이너 파드 배포 → CPU 100% 지속 → **3주 후 AWS 비용 청구서로 최초 인지**. 컨테이너 로그 부재로 공격 경로 파악에 수일 추가 소요. (Tesla 클라우드 환경 크립토재킹 사례, 2018 — RedLock 보고서)

1. 로그 드라이버 로테이션 설정 — 즉시
2. cAdvisor + Prometheus 메트릭 수집 — 즉시
3. CPU/메모리 임계치 알림 설정 — 즉시
4. Falco 런타임 이상행위 탐지 — 단기

1순위 로그 드라이버 로테이션 설정 [즉시]

Docker 기본 로그 드라이버(json-file)는 로테이션 설정이 없으면 로그 파일이 무제한으로 증가한다. `max-size=10m` , `max-file=5` 로 설정하면 로그 파일 총량이 50MB로 제한된다.

/ETC/DOCKER/DAEMON.JSON

```json
{
  "log-driver": "json-file",
  "log-opts": { "max-size": "10m", "max-file": "5" }
}
```

2순위 cAdvisor + Prometheus 메트릭 수집 [즉시]

cAdvisor는 컨테이너별 CPU·메모리·네트워크 I/O를 Prometheus 포맷으로 내보낸다. Prometheus가 수집하고 Grafana에서 시각화하는 조합이 컨테이너 모니터링의 사실상 표준이다.

DOCKER-COMPOSE.YML

```yaml
cadvisor:
  image: gcr.io/cadvisor/cadvisor:v0.49.1
  volumes:
    - /var/run/docker.sock:/var/run/docker.sock:ro          # 아래 주의사항 참고
    - /sys:/sys:ro
    - /var/lib/docker:/var/lib/docker:ro
  ports: ["127.0.0.1:8080:8080"]
```

> **주의사항 — DOCKER.SOCK 모니터링 예외**
> 3.4에서 /var/run/docker.sock 마운트를 금지했으나, cAdvisor처럼 메트릭 수집 목적인 에이전트는 예외적으로 `:ro` (읽기 전용) 마운트를 허용한다. 더 엄격한 환경에서는 Tecnativa docker-socket-proxy를 중간에 두어 GET /containers만 허용하는 것을 권장한다.

3순위 CPU/메모리 임계치 알림 설정 [즉시]

크립토재킹은 CPU 사용률이 지속적으로 높은 수준을 유지하는 패턴이 특징이다. 아래는 80% 초과 상태가 5분 이상 지속될 때 알림을 발송하는 예시 설정이다. 운영 환경의 서비스 특성에 맞춰 임계치를 조정한다.

GRAFANA ALERT PROMQL — CPU 80% 초과 5분 지속

```
avg by (name)(rate(container_cpu_usage_seconds_total{name!=""}[5m])) > 0.8
```

4순위 Falco 런타임 이상행위 탐지 [단기]

Falco는 CNCF 프로젝트로, 리눅스 시스템 콜을 실시간으로 모니터링하여 컨테이너 내부의 이상 행동을 탐지한다. 메트릭 기반 모니터링이 리소스 사용량 이상을 감지한다면, Falco는 프로세스 실행·파일 접근·네트워크 연결 등 행위 수준의 이상을 탐지한다.

FALCO RULE

```yaml
- rule: Terminal shell in container
  condition: spawned_process and container and shell_procs and proc.tty != 0
  output: Shell spawned (user=%user.name container=%container.name)
  priority: WARNING
```

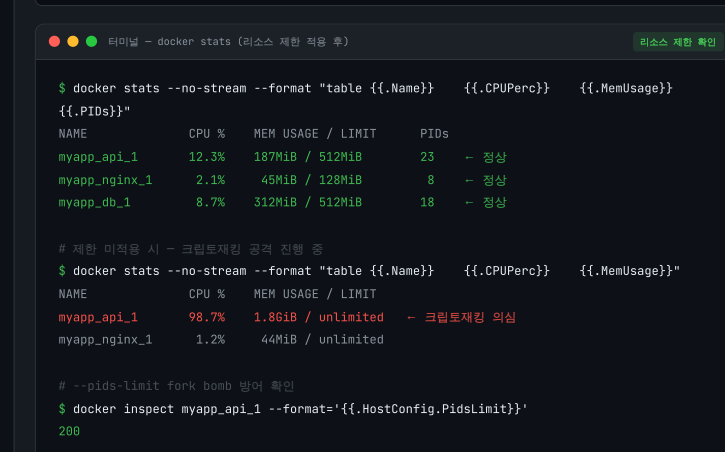
Falco — 런타임 이상행위 탐지 로그 ⚠ 침입 탐지

```
2024-11-01T14:22:11.392814751+0000 Warning Terminal shell in container
  rule:    Terminal shell in container
  message: Shell spawned in a container other than entrypoint
  ─────────────────────────────────────────────────────
  user:      nobody (suspicious — 알 수 없는 유저)
  container: myapp-prod-7d4f9b
  image:     myregistry/myapp:v1.2.3
  process:   /bin/sh (shell 실행 감지)
  cmdline:   sh -i
  proc.tty:  1  ← interactive TTY 감지
  parent:    bash (pid=1337)
  ─────────────────────────────────────────────────────
  → Slack #security-alerts 웹훅 알림 발송됨
  → SIEM 이벤트 전송됨 (severity: HIGH)
```

> **주의사항**
> 로그에 패스워드, API 키, 개인정보가 출력되지 않도록 앱 수준에서 마스킹 처리 필요. 중앙 로그 수집 시스템이 민감정보 저장소가 될 수 있다.

#6 순위 `OWASP Top 10 D07` 리소스 제한 — Medium CIS §5

리소스 제한이 없으면 컨테이너 하나가 호스트 전체 CPU·메모리를 독점할 수 있다.



--cpus=0.5 제한 적용 전/후 CPU 점유 비교 (크립토재킹 시나리오)

1. CPU 제한 (--cpus) — 즉시
2. 메모리 제한 (--memory) — 즉시
3. 프로세스 수 제한 (--pids-limit) — 즉시

1순위 CPU 제한 (--cpus) [즉시]

`--cpus=0.5` 는 컨테이너 CPU 사용량을 1코어의 50%로 제한한다. 운영 전에 `docker stats` 로 실사용량을 측정한 뒤, 평균 사용량의 2배를 초기 limit 기준점으로 설정하는 것을 권장한다.

SHELL

```
$ docker run --cpus=0.5 myapp
$ docker stats --format 'table {{.Name}}        {{.CPUPerc}}     {{.MemUsage}}'
```

2순위 메모리 제한 (--memory) [즉시]

`--memory=512m` 으로 메모리를 512MB로 제한한다. 초과하면 OOMKilled로 재시작되면서 호스트 메모리 고갈을 막는다.

SHELL

```
$ docker run \
  --memory=512m \
  --memory-swap=512m \          # memory+swap 합산 한도 = 512m → 사실상 swap 0
  --memory-reservation=256m \
  myapp
```

3순위 프로세스 수 제한 (--pids-limit) [즉시]

fork bomb 공격은 프로세스를 무한 복제하여 PID 자원을 고갈시킨다. `--pids-limit=200` 을 설정하면 컨테이너 내 프로세스 수가 제한되어 호스트 전체로 영향이 확산되지 않는다.

DOCKER-COMPOSE.YML

```yaml
deploy:
  resources:
    limits:       { cpus: "0.5", memory: 512M }
    reservations: { cpus: "0.1", memory: 128M }
  pids_limit: 200          # cgroup 기반 PID 제한
```

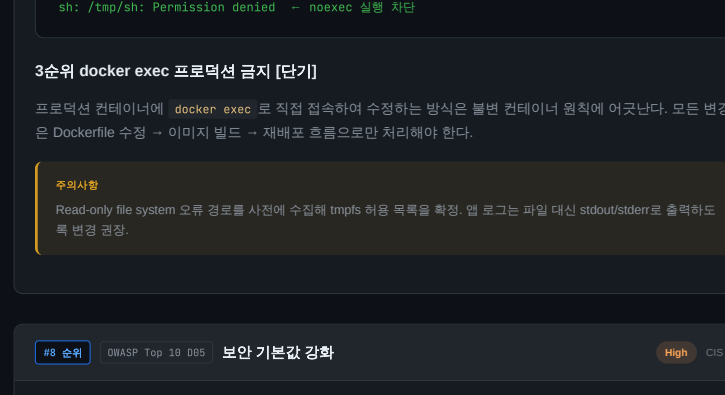
터미널 — docker stats (리소스 제한 적용 후) 리소스 제한 확인

```
$ docker stats --no-stream --format "table {{.Name}}    {{.CPUPerc}}    {{.MemUsage}}
{{.PIDs}}"
NAME             CPU %    MEM USAGE / LIMIT     PIDs
myapp_api_1      12.3%    187MiB / 512MiB       23    ← 정상
myapp_nginx_1     2.1%     45MiB / 128MiB        8    ← 정상
myapp_db_1        8.7%    312MiB / 512MiB       18    ← 정상

# 제한 미적용 시 — 크립토재킹 공격 진행 중
$ docker stats --no-stream --format "table {{.Name}}    {{.CPUPerc}}    {{.MemUsage}}"
NAME             CPU %    MEM USAGE / LIMIT
myapp_api_1      98.7%    1.8GiB / unlimited   ← 크립토재킹 의심
myapp_nginx_1     1.2%     44MiB / unlimited

# --pids-limit fork bomb 방어 확인
$ docker inspect myapp_api_1 --format='{{.HostConfig.PidsLimit}}'
200
```

> **주의사항**
> OOMKilled 반복 발생 시 limit 상향 필요. 통상 평균 사용량의 2배를 limit으로 설정. 참고: `deploy.resources` 는 Docker Compose v3(Swarm mode) 전용 문법이다. 단독 Compose 환경에서는 `cpus` , `mem_limit` , `pids_limit` 키를 서비스 레벨에서 직접 사용한다.

#7 순위 `OWASP Top 10 D09` 불변 컨테이너 원칙 — Medium CIS §5

파일시스템에 쓰기가 가능하면 RCE 이후 공격자가 바이너리 교체·크론탭 수정·웹셸 설치로 지속성(Persistence)을 확보할 수 있다.



쓰기 가능 vs --read-only 파일시스템 — 공격자 지속성 확보 차단

1. --read-only 파일시스템 적용 — 즉시
2. tmpfs로 쓰기 경로 최소 허용 — 즉시
3. docker exec 프로덕션 금지 — 단기

1순위 --read-only 파일시스템 적용 [즉시]

`--read-only` 는 컨테이너 루트 파일시스템 전체를 읽기 전용으로 마운트한다. RCE 발생 후 공격자가 바이너리를 교체하거나 웹셸을 설치하는 것을 원천적으로 차단한다.

SHELL

```
$ docker run --read-only myapp
$ docker run myapp 2>&1 | grep 'Read-only file system'          # 쓰기 경로 사전 파악
```

2순위 tmpfs로 쓰기 경로 최소 허용 [즉시]

`--tmpfs` 로 특정 경로만 메모리 기반 파일시스템으로 마운트하면 마운트된 경로에서만 쓰기가 허용된다. `noexec` 옵션을 함께 지정하면 해당 경로에 업로드된 파일의 실행도 차단된다.

SHELL ✅ Best Practice

```
$ docker run \
  --read-only \
  --tmpfs /tmp:rw,noexec,nosuid,size=64m \          # noexec: 업로드 파일 실행 차단
  --tmpfs /var/run:rw,noexec,nosuid \
  myapp
```

터미널 — --read-only 파일시스템 검증 불변 파일시스템 확인

```
# ❌ 기본 컨테이너 — 바이너리 교체 가능 (공격자 악용)
$ docker run --rm node:22-alpine sh -c "echo 'malware' > /usr/bin/node && echo ok"
ok  ← 바이너리 교체 성공 (위험!)

# ✅ --read-only 적용 후 — 모든 쓰기 차단
$ docker run --rm --read-only myapp sh -c "echo 'malware' > /usr/bin/node"
sh: can't create /usr/bin/node: Read-only file system

# ✅ /tmp는 tmpfs로 쓰기 허용, noexec으로 실행은 차단
$ docker run --rm --read-only --tmpfs /tmp:rw,noexec myapp sh -c "echo test > /tmp/a && ca
t /tmp/a"
test  ← /tmp 쓰기 허용됨
$ docker run --rm --read-only --tmpfs /tmp:rw,noexec myapp sh -c "cp /bin/sh /tmp/sh && /t
mp/sh"
sh: /tmp/sh: Permission denied  ← noexec 실행 차단
```

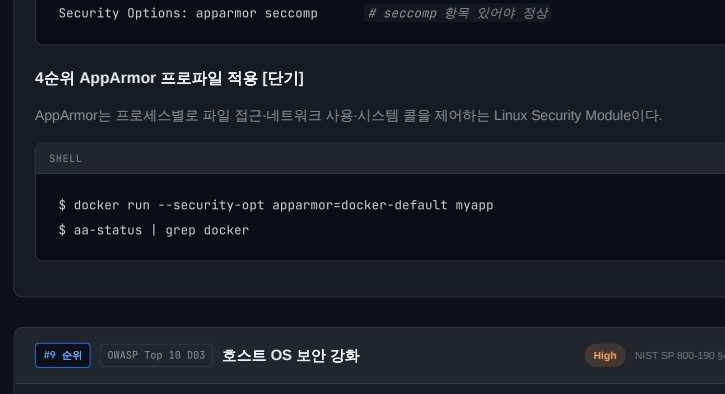
3순위 docker exec 프로덕션 금지 [단기]

프로덕션 컨테이너에 `docker exec` 로 직접 접속하여 수정하는 방식은 불변 컨테이너 원칙에 어긋난다. 모든 변경은 Dockerfile 수정 → 이미지 빌드 → 재배포 흐름으로만 처리해야 한다.

> **주의사항**
> Read-only file system 오류 경로를 사전에 수집해 tmpfs 허용 목록을 확정. 앱 로그는 파일 대신 stdout/stderr로 출력하도록 변경 권장.

#8 순위 `OWASP Top 10 D05` 보안 기본값 강화 — High CIS §5

`--privileged` 는 모든 Linux capability를 부여하고 Seccomp·AppArmor 프로파일을 비활성화한다. 컨테이너 격리를 완전히 해제하는 것과 동일한 효과를 가진다.



--privileged vs --cap-add — 보안 격리 비교

1. --privileged 완전 금지 — 즉시
2. daemon.json 보안 기본값 설정 — 즉시
3. Seccomp 기본 프로파일 확인 — 즉시
4. AppArmor 프로파일 적용 — 단기

1순위 --privileged 완전 금지 [즉시]

`--privileged` 는 모든 Linux capability를 부여하고, cgroup/namespace 제한을 해제하며, /dev 아래 호스트 디바이스 전체에 대한 접근을 허용한다. 컨테이너 격리가 완전히 무력화된다. 특정 capability만 필요하다면 `--cap-add` 로 해당 capability만 따로 추가하는 방식으로 대체하자. CI에서 Compose 파일에 privileged가 있는지 자동으로 검증하는 스크립트를 포함하는 것을 권장한다.

SHELL

```
# ❌ Bad Practice — 모든 보안 격리 무력화
$ docker run --privileged myapp

# ✅ Best Practice — 필요한 capability만
$ docker run --cap-add SYS_PTRACE myapp

# CI: privileged 사용 자동 검증
$ grep -r 'privileged.*true' docker-compose*.yml && exit 1
```

2순위 daemon.json 보안 기본값 설정 [즉시]

`no-new-privileges: true` 는 전역 권한 상승을 막고, `userns-remap` 은 컨테이너 내부의 root(UID 0)를 호스트의 비권한 UID(예: 100000)에 매핑한다. 컨테이너가 탈출하더라도 호스트에서 일반 유저 권한으로만 동작하게 된다. 단, NFS·일부 볼륨 마운트와 호환성 문제가 발생할 수 있어 운영 환경에서는 사전 검증이 필요하다.

/ETC/DOCKER/DAEMON.JSON

```
{ "no-new-privileges": true, "userns-remap": "default" }
$ systemctl restart docker
```

3순위 Seccomp 기본 프로파일 확인 [즉시]

Docker는 기본 Seccomp 프로파일을 통해 300개 이상의 시스템 콜 중 44개를 차단한다.[5]

SHELL

```
$ docker info | grep -i seccomp
Security Options: apparmor seccomp          # seccomp 항목 있어야 정상
```

4순위 AppArmor 프로파일 적용 [단기]

AppArmor는 프로세스별로 파일 접근·네트워크 사용·시스템 콜을 제어하는 Linux Security Module이다.

SHELL

```
$ docker run --security-opt apparmor=docker-default myapp
$ aa-status | grep docker
```

#9 순위 `OWASP Top 10 D03` 호스트 OS 보안 강화 — High NIST SP 800-190 §4.3

컨테이너는 호스트 커널을 공유하기 때문에, 호스트 커널에 취약점이 존재하면 컨테이너 수준의 격리 설정과 무관하게 보안 경계가 우회될 수 있다.

> **실제 사례 — CVE-2016-5195 (DIRTYCOW)**
> 리눅스 커널 2.6.22~4.8.3 영향. 컨테이너 내부에서 호스트 /etc/passwd 수정 및 root 계정 추가 가능. 즉각 전 세계 긴급 패치 배포.

1. Docker Engine 자동 업데이트 — 단기
2. 호스트 OS 커널 자동 패치 — 단기
3. Docker Rootless 모드 전환 — 중기
4. 컨테이너 전용 OS 사용 — 중기

1순위 Docker Engine 자동 업데이트 [단기]

Docker Engine(moby)과 runc는 보안 취약점 발견 시 즉시 패치 버전이 배포된다. CVE-2019-5736(runc)[8]처럼 컨테이너 탈출로 직결되는 취약점은 패치 배포 즉시 업데이트하지 않으면 실제 공격에 노출될 수 있다. `unattended-upgrades` 에 `docker-ce` 를 포함하여 자동으로 적용되도록 설정하는 것을 권장한다.

SHELL

```
$ apt-get update && apt-get install --only-upgrade docker-ce
$ docker version --format '{{.Server.Version}}'
```

2순위 호스트 OS 커널 자동 패치 [단기]

컨테이너가 공유하는 커널에 취약점이 있으면 모든 컨테이너 보안 설정이 무의미해질 수 있다. Ubuntu에서는 unattended-upgrades 패키지로 보안 패치를 자동 적용할 수 있으며, RHEL/CentOS에서는 dnf-automatic 서비스를 사용한다.

SHELL

```
# Ubuntu
$ apt-get install -y unattended-upgrades && dpkg-reconfigure unattended-upgrades

# RHEL / CentOS
$ dnf install -y dnf-automatic && systemctl enable --now dnf-automatic.timer
```

3순위 Docker Rootless 모드 전환 [중기]

Rootless 모드는 daemon과 컨테이너 전체를 일반 유저 네임스페이스 내에서 실행한다. §3.2에서 다룬 컨테이너 수준의 non-root 실행과 달리, 이 방식은 Docker daemon 자체의 권한을 제한하여 daemon 수준 취약점 발생 시에도 호스트 root 권한이 노출되지 않도록 한다.

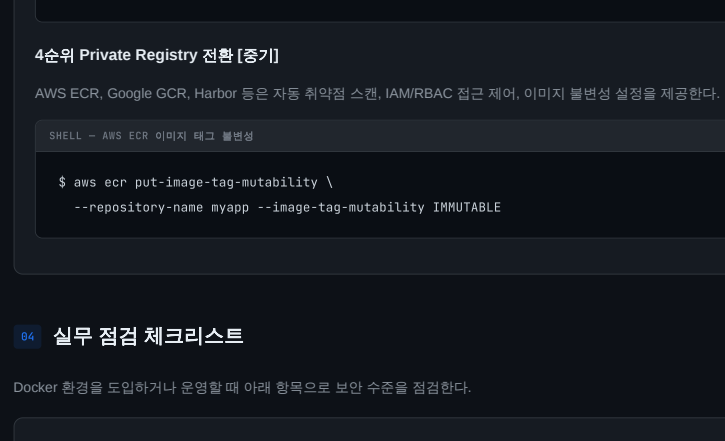
설치 명령어 및 상세 설정은 **3.2 최소 권한 실행 원칙 > 4순위**를 참고한다.

4순위 컨테이너 전용 OS 사용 [중기]

Container-Optimized OS(GCP), Bottlerocket(AWS), Flatcar Linux는 컨테이너 실행만을 위해 설계된 최소화 OS다. 루트 파일시스템이 읽기 전용으로 마운트되어 OS 자체에 대한 변조 공격이 차단된다.

#10 순위 `OWASP Top 10 D02` 공급망 보안 — Medium CIS §4.2

신뢰하던 베이스 이미지나 패키지 자체가 오염되는 공급망 공격이 실제로 발생하고 있다. SBOM(Software Bill of Materials)과 이미지 서명을 활용하여 공급망 전체의 무결성을 검증한다.[1]



공급망 공격 흐름과 각 단계별 방어 지점

> **실제 사례 — CODECOV 공급망 공격 (2021)**
> CI/CD bash 스크립트에 악성 코드 삽입 → 실행 환경의 환경변수 전체(AWS 키, GitHub 토큰) 탈취. 영향 기업 **18,000개 이상**.

1. digest 고정 참조 — 즉시
2. SBOM 생성 및 정기 취약점 스캔 — 단기
3. 이미지 서명 및 검증 (Cosign) — 단기
4. Private Registry 전환 — 중기

1순위 digest 고정 참조 [즉시]

이미지 태그는 가변적이다. digest(`@sha256:...`)는 콘텐츠 기반의 변경 불가능한 식별자로, 특정 이미지 레이어의 사용을 보장한다.

DOCKERFILE / SHELL

```
$ docker inspect --format='{{index .RepoDigests 0}}' node:22-alpine
FROM node:22-alpine@sha256:a1b2c3d4e5...
```

2순위 SBOM 생성 및 정기 취약점 스캔 [단기]

SBOM(Software Bill of Materials)을 생성해 두면 신규 CVE 발표 시 이미지를 재빌드하지 않고도 영향 범위를 즉시 파악할 수 있다.[10]

SHELL

```
$ syft myapp:latest -o spdx-json > sbom.json
$ grype sbom:./sbom.json --fail-on high
```

3순위 이미지 서명 및 검증 (Cosign) [단기]

Cosign은 Sigstore 프로젝트의 이미지 서명 도구다. Kubernetes Policy Controller와 연동하면 미서명 이미지의 클러스터 배포를 자동 거부한다.

SHELL

```
$ cosign generate-key-pair
$ cosign sign --key cosign.key myregistry/myapp:$(git rev-parse --short HEAD)
$ cosign verify --key cosign.pub myregistry/myapp:abc1234
```

4순위 Private Registry 전환 [중기]

AWS ECR, Google GCR, Harbor 등은 자동 취약점 스캔, IAM/RBAC 접근 제어, 이미지 불변성 설정을 제공한다.

SHELL — AWS ECR 이미지 태그 불변성

```
$ aws ecr put-image-tag-mutability \
  --repository-name myapp --image-tag-mutability IMMUTABLE
```

04 실무 점검 체크리스트

Docker 환경을 도입하거나 운영할 때 아래 항목으로 보안 수준을 점검한다.

이미지
- ☐ 최소화 이미지 사용 + 구체적 버전 고정 (latest 금지)
- ☐ CI/CD Trivy 스캔 통합, HIGH 이상 CVE 발견 시 빌드 차단
- ☐ DOCKER_CONTENT_TRUST=1 또는 Cosign 서명 검증 적용

권한
- ☐ Dockerfile non-root USER 지시어 명시 + COPY --chown 설정
- ☐ 런타임 --cap-drop ALL --no-new-privileges 적용
- ☐ /etc/docker/daemon.json에 no-new-privileges: true 전역 설정

시크릿
- ☐ Dockerfile ENV/ARG에 시크릿 하드코딩 없음
- ☐ .dockerignore에 .env, *.key, *.pem, secrets/ 추가
- ☐ BuildKit --secret 또는 Docker Secrets로 시크릿 주입

네트워크
- ☐ frontend/app/data 3계층 네트워크 분리 + internal: true
- ☐ 모든 포트 바인딩 127.0.0.1 사용, DB/Redis 외부 포트 미노출
- ☐ /var/run/docker.sock 컨테이너 마운트 없음 확인

리소스 / 불변성 / 기본값
- ☐ --cpus, --memory, --pids-limit 제한 설정
- ☐ --read-only 적용 + 쓰기 필요 경로만 tmpfs 허용
- ☐ --privileged 사용 없음 + CI 검증 스크립트 포함

모니터링 / 호스트 / 공급망
- ☐ 로그 드라이버 max-size=10m, max-file=5 설정
- ☐ cAdvisor + Prometheus + Grafana 구성, CPU·메모리 임계치 알림 설정
- ☐ Docker Engine 최신 버전 + 호스트 OS 자동 보안 패치
- ☐ SBOM 생성 + 신규 CVE 발표 시 영향 범위 점검 프로세스

05 참고 문헌

[1] OWASP Docker Top 10
owasp.org/www-project-docker-top-10/
활용: 우선순위 체계 및 D01~D10 항목 분류

[2] OWASP Docker Security Cheat Sheet
cheatsheetseries.owasp.org
활용: --cap-drop, --no-new-privileges, 네트워크 격리 실무 적용

[3] CIS Docker Benchmark v1.8.0
cisecurity.org/benchmark/docker
활용: daemon.json 보안 기본값, no-new-privileges 전역 설정

[4] NIST SP 800-190: Application Container Security Guide
csrc.nist.gov
활용: 컨테이너 위협 모델 및 호스트 OS 보안 강화

[5] Docker Official Security Docs
docs.docker.com/engine/security/
활용: Rootless 모드, BuildKit --secret, /var/run/docker.sock 격리

[6] Snyk: Node.js Docker Image Analysis (2024)
snyk.io
활용: node:latest vs alpine CVE 수·이미지 크기 비교 수치

[7] Sysdig 2024 Cloud-Native Security Report
sysdig.com
활용: 컨테이너 보안 사고 주요 원인 통계 (misconfiguration, 과도한 권한)

[8] CVE-2019-5736 (runc)
cve.mitre.org
활용: root 컨테이너 탈출 시나리오 및 non-root 적용 근거

[9] Sigstore / Cosign
docs.sigstore.dev
활용: 이미지 서명·검증 명령어 (3순위 공급망 보안)

[10] Anchore Syft SBOM Generator
github.com/anchore/syft
활용: SBOM 생성 및 CVE 영향 범위 즉시 파악